



Informe Técnico de Entrega Final

Reconstrucción Integral Full Stack (Sprint 3 y 4) · Arquitectura Cliente-Servidor

Desarrollador: Matías Carlsson	Institución: ITBA / Santander
Repositorio GitHub: github.com/MatiasCarlsson/Full-Stack-ITBA	Rama Activa: <code>matias</code>
Pull Request a dev: PR #1 (Merged / Fusionada)	Pull Request a main: PR #2 (Abierta para Release)
Entornos Locales: Backend en :3001 · Frontend en :3000	Fecha de Entrega: Octubre 2026

1. Resumen Ejecutivo del Proyecto

El presente informe formaliza la entrega de los **Sprints 3 y 4** del programa Full Stack Developer. El objetivo primordial consistió en la **transformación integral de una maqueta web estática hacia una verdadera aplicación cliente-servidor**, estructurada sobre dos pilares independientes pero estrechamente integrados: una **API REST** desarrollada con **Node.js y Express** para servir los datos del catálogo, y un **frontend interactivo** construido desde cero en **React 19** que consume dichos servicios mediante peticiones HTTP asincrónicas (fetch).

El desarrollo respeta al 100% las especificaciones de diseño y el *Manual de Marca* de **Hermanos Jota** (mueblería de autor con madera nativa y acabados nobles). La interfaz prescinde completamente de emojis en favor de un sistema de **iconografía SVG minimalista**, incorpora botones de acción unificados ("**Agregar**" + icono SVG de carrito), implementa el favicon oficial de la marca en el navegador y cumple con estándares estrictos de pruebas unitarias y accesibilidad WCAG 2.1.

2. Arquitectura de la Solución y Stack Tecnológico

La solución se organizó bajo una arquitectura modular limpia desacoplada en dos directorios de trabajo:

Capa / Módulo	Tecnología	Puerto / Configuración	Responsabilidad Principal
Backend API /backend	Node.js · Express 4 CORS · Nodemon	<code>http://localhost:3001</code>	Expone endpoints REST modulares con <code>express.Router</code> , middleware de logging con timestamp y catálogo en archivo local.
Frontend UI /client	React 19 · CRA Testing Library · CSS	<code>http://localhost:3000</code> Proxy a :3001	Interfaz SPA con renderizado condicional, estado global de carrito en <code>App.js</code> , control asincrónico (carga, error, reintento) y diseño editorial.
Calidad & Tests	Jest · RTL · ESLint Gitflow · Commits	5/5 tests pasados 0 errores ESLint	Suite automatizada de pruebas para marca, navegación, tarjetas, detalle y contacto, cumpliendo reglas estrictas de Testing Library.

3. Matriz de Cumplimiento Técnico (Consigna Oficial)

Verificación exhaustiva de cada uno de los ítems evaluados en el Sprint 3 y 4:

Requisito Exigido	Área	Archivo de Implementación	Estado
Datos de productos en archivo .js local (array de objetos)	Backend	backend/src/data/productos.js	CUMPLE 100%
GET /api/productos con listado completo en JSON	Backend	backend/src/routes/productos.js	CUMPLE 100%
GET /api/productos/:id con producto o 404 estructurado	Backend	backend/src/routes/productos.js	CUMPLE 100%
Middleware global de logging (método HTTP y URL con timestamp)	Backend	backend/src/middlewares/logger.js	CUMPLE 100%
Middleware express.json() para futuras peticiones POST	Backend	backend/src/index.js	CUMPLE 100%
Organización modular con express.Router	Backend	backend/src/routes/productos.js	CUMPLE 100%
6 componentes React: Navbar, Footer, ProductCard, ProductList, ProductDetail, ContactForm	Frontend	client/src/components/*.jsx	CUMPLE 100%
Fetch asíncronico a /api/productos con estados de carga y error	Frontend	client/src/components/ProductList.jsx	CUMPLE 100%
Renderizado de lista con .map() y keys únicas basadas en ID	Frontend	client/src/components/ProductList.jsx	CUMPLE 100%
Detalle de producto mediante renderizado condicional	Frontend	client/src/App.js	CUMPLE 100%
Carrito como estado global en App.js y contador en Navbar vía props	Frontend	client/src/App.js y Navbar.jsx	CUMPLE 100%
Formulario de contacto controlado con validación estricta y feedback	Frontend	client/src/components/ContactForm.jsx	CUMPLE 100%

4. Aspectos Técnicos Destacados de la Implementación

4.1. Backend API (Node.js + Express)

- **Router Modular:** Los endpoints se aislaron en routes/productos.js y se montaron bajo app.use('/api/productos', productosRouter).
- **Manejo de Errores 404:** Búsquedas por ID inexistente (ej: GET /api/productos/999) responden status 404 con JSON descriptivo: {"error": "Producto no encontrado", "mensaje": "No existe un producto con el ID 999"}.
- **Logging en Consola:** Middleware personalizado que imprime fecha/hora ISO, método y URL de cada request.
- **Assets Sanitizados:** Servidor de estáticos en /imagenes con nombres kebab-case sin tildes ni espacios para evitar fallos de proxy.

4.2. Frontend Reactivo (React 19)

- **Consumo Asíncronico Limpio:** fetch('/api/productos') respaldado por AbortController para cancelar peticiones pendientes al desmontar el componente (evitando memory leaks).
- **Ciclo de Vida Completo:** Spinner interactivo durante la carga, pantalla de error amigable con botón de reintento en caídas de red y renderizado dinámico en éxito.
- **Carrito Global & Drawer:** Estado en App.js con acumulación inteligente por ID. Panel lateral deslizable (*Cart Drawer*) para sumar (+), restar (-), eliminar piezas y vaciar carrito con cálculo de subtotal en ARS.
- **Renderizado Condicional:** Navegación instantánea entre Catálogo, Detalle de Producto y Formulario de Contacto sin recargas de página.

5. Identidad Visual, UI/UX y Sistema de Diseño

- **Manual de Marca:** Tokens CSS con la paleta oficial: Siena Tostado (#A0522D), Verde Salvia (#87A96B), Alabastro (#F5E6D3) y Vara de Oro (#D4A437).
- **Erradicación Total de Emojis:** Reemplazados al 100% por un catálogo de iconos vectoriales SVG limpios y arquitectónicos en Icons.jsx.
- **Botón de Compra Unificado:** En tarjetas y detalle, el botón de agregar presenta estrictamente "Agregar" + el SVG reutilizable <CartIcon />.
- **Branding en Navegador:** Favicon oficial en formato .ico multirresolución y .svg vectorial, junto a manifest.json y logos PWA.
- **Accesibilidad WCAG:** :focus-visible de alto contraste, roles semánticos y soporte para prefers-reduced-motion.

6. Calidad de Código y Pruebas Unitarias (5/5 PASS)

Suite automatizada implementada con **React Testing Library** en client/src/App.test.js:

Caso de Prueba (Test Case)	Componente	Criterio Verificado	Resultado
1. Renderizado de Marca	App.js	Presencia del nombre Hermanos Jota en la portada.	PASS ✓
2. Navegación y Carrito	Navbar.jsx	Botones accesibles de catálogo, contacto y carrito.	PASS ✓
3. Botón Compra en Card	ProductCard.jsx	Texto "Agregar", etiqueta accesible y presencia del SVG de carrito.	PASS ✓
4. Botón Compra en Detalle	ProductDetail.jsx	Texto "Agregar", etiqueta accesible y presencia del SVG de carrito.	PASS ✓
5. Formulario de Contacto	ContactForm.jsx	Inputs para nombre, email, mensaje y botón de envío.	PASS ✓

Auditoría ESLint: Se eliminaron accesos directos al DOM resolviendo la regla testing-library/no-node-access con within(el).getByTestId("cart-icon"). **0 errores, 0 advertencias.**

7. Flujo de Ramas Git y Pull Requests en GitHub

Rama / Pull Request	Propósito y Contenido	Estado en GitHub
Rama matias (Rama de Autor)	Commits semánticos con la implementación completa del backend, client, tests y documentación.	Pushed & Active Sincronizada
Pull Request #1 matias → dev	Integración formal de los cambios de desarrollo a la rama staging.	MERGED ✓ Fusionada en GitHub
Pull Request #2 dev → main	Release final del Sprint 3 y 4 hacia la rama principal de producción.	OPEN (Lista) PR #2 creada

8. Guía de Instalación y Ejecución

1. Iniciar Backend (API REST en :3001): cd backend && npm install && npm run dev
2. Iniciar Frontend (React en :3000): cd client && npm install && npm start
3. Ejecutar Pruebas Automatizadas: cd client && npm test -- --watchAll=false

Desarrollador: Matías Carlsson	Evaluación Final Sprint 3 y 4:
Curso: Full Stack Web Developer — ITBA / Santander	Cumplimiento Total (100%) · Aprobado
Repositorio: https://github.com/MatiasCarlsson/Full-Stack-ITBA	Listo para entrega formal